

02 语法差异速览

目标:看完能用 Java 写基本语法,不踩 C++ 选手第一周必踩的语法雷。

范围:只讲刷题会碰到的纯语法(引用 / 字符串不可变 / 数组 / final / static / 类与对象 / char vs String / 异常 / for-each),装箱和 lambda 已经在 03 / 04 讲。

C++ 类比:本篇主要做"翻译",不展开 Java 完整语法。

一、引用 vs C++ 指针(第一个要改的肌肉记忆)

C++ 选手写 Java 最大的不适应就是没有 `\*` / `&` / `->` 那一套。Java 引用就是"指向对象的句柄",语法上几乎等价于 C++ 的"普通对象"(p.name 不是 p->name)。

1.1 核心差异

维度	C++	Java
声明	<code>Person* p;</code>	<code>Person p;</code>
创建	<code>p = new Person();</code>	<code>p = new Person();</code>
访问成员	<code>p->name</code> 或 <code>(*p).name</code>	<code>p.name</code> (只有 `.`)
判空	<code>if (p == nullptr)</code>	<code>if (p == null)</code>
销毁	<code>delete p;</code>	不用管(JVM 垃圾回收)
指针运算	<code>p + 1`、`*p = ...`</code>	没有指针运算

1.2 最小代码

```
// Java:引用 p 指向堆上的 Person 对象
Person p = new Person();
p.name = "Tom";          // 直接用 . 访问,没有 ->
System.out.println(p.name); // Tom
```

```
// C++ 对照
Person* p = new Person();
p->name = "Tom";          // 要么 ->,要么 (*p).name
std::cout << p->name;
```

1.3 关键提醒

- Java 引用默认是 `null` (对比 C++ 是野指针): `Person p;` 此时 `p` 是 `null`,直接 `p.name` 会抛 `NullPointerException`。

- 没有 `delete`: 对象不用手动释放,GC 帮你收。但别把"不用管"理解成"想怎么写都行"——C++ 选手最常踩的坑是漏掉 `new`,让引用保持 `null`。

- 没有 `\*&` 取引用: Java 函数参数传引用类型,函数内改的就是原对象(和 C++ 传指针类似),不存在"传值 vs 传引用"的语法选择——所有引用类型传的都是引用副本(指向同一对象)。

二、字符串不可变(C++ 选手第 1 雷)

Java 的 String 是不可变对象——这意味着你永远改不了字符串本身,所有"改"都是创建新对象 + 引用重新指向。

2.1 核心事实

```
String s = "abc";      // s 指向对象 "abc"
s += "def";            // 不是改原对象!而是创建新对象 "abcdef",s 重新指向它
System.out.println(s); // "abcdef"
```

真相:+= 运算符在编译期被翻译成 `s = new StringBuilder().append(s).append("def").toString();`,旧的 "abc" 对象还在堆里(等 GC),s 换了个指向。

2.2 为什么这么设计

- 哈希稳定:String 经常作为 HashMap / HashSet 的 key(见 03 装箱小框里讲过 key 必须不可变),不可变才保证 hashCode 永远不变。
- 线程安全:多线程共享同一个 String 不需要同步。
- 字符串常量池:相同字面量共享一个对象(String s1 = "abc"; String s2 = "abc"; s1 和 s2 指向同一对象),节省内存。

2.3 错误 vs 正确

```
// 错误:以为 s[0] = 'X' 能改字符,实际 String 没有 [] 运算符
String s = "hello";
s[0] = 'X';           // 编译报错
```

```
// 正确 1:用 StringBuilder(可变字符串,刷题推荐)
StringBuilder sb = new StringBuilder("hello");
sb.setCharAt(0, 'X');    // "Xello"
String s = sb.toString(); // 转回 String
```

```
// 正确 2:用 char[] 数组(刷题偶尔用)
char[] arr = "hello".toCharArray();
arr[0] = 'X';           // 直接改
String s = new String(arr); // 转回 String
```

2.4 C++ 对照

```
// C++ 的 string 是字符数组的封装,可以直接改
std::string s = "hello";
s[0] = 'X';           // "Xello",合法
s += "def";           // 原地追加,合法
```

2.5 记忆口诀

String 不可变,改用 StringBuilder。StringBuilder 的 API(append / reverse / deleteCharAt / setCharAt / toString)在 04 工具类详讲,本篇只建立"不可变"的认知。



三、数组定长 vs `vector`

Java 数组长度固定,创建时多大就多大,不能 `push_back` / `pop_back`。要动态数组用 `ArrayList<Integer>`(03 容器小节详讲)。

3.1 核心差异

维度	C++ `vector<int>`	Java `int[]`	Java `ArrayList<Integer>`
长度	动态(可 <code>push_back</code>)	定长(创建后不可变)	动态(可 <code>add</code>)
取长度	<code>v.size()</code>	<code>arr.length</code> (无括号!)	<code>list.size()</code>
末尾添加	<code>v.push_back(x)</code>	不支持	<code>list.add(x)</code>
按下标访问	<code>v[i]</code>	<code>arr[i]</code>	<code>list.get(i)</code>

长度语法不同:`arr.length`(数组,字段)/ `s.length()`(字符串,方法)/ `list.size()`(集合,方法)——C++ 全靠 `v.size()`。这个坑在 06 第 12 坑再深讲。

3.2 错误 vs 正确:想给数组追加元素

```
// 错误:Java 数组定长,没有 push_back
int[] arr = {1, 2, 3};
arr.push_back(4); // 编译报错,Java 数组没这方法
arr[3] = 4; // 运行时 ArrayIndexOutOfBoundsException
```

```
// 正确 1(刷题不推荐,繁琐):用 Arrays.copyOf 复制 + 扩容
int[] arr = {1, 2, 3};
arr = Arrays.copyOf(arr, arr.length + 1); // 复制成新数组,长度 +1
arr[3] = 4; // 现在 arr 长度是 4

// 正确 2(刷题推荐):用 ArrayList
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
list.add(4); // 动态添加,舒服
int[] arr = list.stream().mapToInt(Integer::intValue).toArray(); // 转回 int[]
```

3.3 C++ 对照

```
// C++ 一个 vector 搞定
std::vector<int> v = {1, 2, 3};
v.push_back(4); // 动态添加
std::cout << v.size(); // 4
```

3.4 选型判断

- 长度已知 / 性能敏感:`int[]` 数组(比如 LeetCode 1 两数之和接收 `int[] nums`)
- 长度未知 / 动态增删:`ArrayList<Integer>`(见 03)
- 刷题 90% 场景用 `ArrayList`,因为"长度未知"是常态

四、`final` 关键字



C++ 的 `const` 只能修饰变量。Java 的 `final` 三种位置都能修饰,语义各有不同。

4.1 三种用法

位置	含义	C++ 对应
<code>final int x = 5;</code>	变量只能赋值一次(常量)	<code>const int x = 5;</code>
<code>final int[] a = {1,2,3};</code>	引用不可变(<code>a</code> 不能指向新数组),但 <code>a[0] = 99</code> 可以改	无完全对应
<code>final void foo() {}</code>	方法不可被子类重写	C++ 没有这个(有 <code>virtual</code> 关键字相反用法)
<code>final class Foo {}</code>	类不可被继承	C++ 没有

4.2 关键:final 引用 ≠ 不可变内容

C++ 选手最容易栽这里。`final` 修饰引用本身(指针指向),不修饰指向的对象内容。

```
// 错误理解:以为 final 数组完全不能改
final int[] a = {1, 2, 3};
a = new int[] {4, 5, 6}; // 编译报错!a 引用不可变,不能指向新数组
a[0] = 99;              // 合法!改的是数组内容,引用没变
System.out.println(a[0]); // 99
```

```
// C++ 没这个语法,const int* 修饰的是"指向常量",也不是一回事
const int* p = new int[3]{1,2,3};
p[0] = 99; // 错误,p 指向常量
```

4.3 刷题常见用法

- 算法题里 `final` 几乎不用——主要在 `lambda` 捕获变量时需要"事实上 `final`"(本篇不讲 `lambda`,见 04 Comparator)。
- 唯一会碰的:LeetCode 题目里有时要传 `final int[]` 给内部类,加上就行,不用纠结。

五、`static` 关键字(C++ 选手易混)

C++ `static` 在不同位置语义完全不同(局部变量 = 延长生命周期,文件作用域 = 文件内可见,类成员 = 类共享)Java `static` 只对应类成员这一种。

5.1 核心差异

C++ `static` 位置	含义	Java 对应
函数内局部变量	延长生命周期(整个程序)	无对应
文件作用域	文件内可见	无对应
类的成员变量 / 方法	类共享(所有实例共用)	<code>static` 成员(同 C++)</code>

5.2 刷题常见用法

全局链表头 / 全局计数器:用 `static` 修饰,所有方法共享。



```
// 刷题常见:全局链表头,LeetCode 题解里经常这样写
class ListNode {
    int val;
    ListNode next;
    static ListNode head;    // 静态字段,所有 ListNode 实例共享
    ListNode(int v) { val = v; }
}

// 初始化(通常在 main 或 Solution 构造里)
ListNode.head = new ListNode(0); // 全局入口
```

5.3 最小代码

```
public class Counter {
    static int count = 0;    // 静态字段:所有 Counter 实例共享

    Counter() {
        count++;            // 每次 new 都 +1
    }

    static void print() {    // 静态方法:不依赖实例就能调用
        System.out.println(count);
    }

    public static void main(String[] args) {
        new Counter();
        new Counter();
        new Counter();
        Counter.print();    // 3.不用 new 也能调静态方法
    }
}
```

```
// C++ 对照(类的 static 成员,跟 Java 语义一样)
class Counter {
public:
    static int count;
    Counter() { count++; }
};

int Counter::count = 0;    // C++ 要在类外单独定义,Java 不用
```

5.4 关键区别

- Java 静态字段不用在类外定义:C++ 写 `int Counter::count = 0;` 是必须的,Java 写在类里就行。
- Java 静态方法不能用 `this`:C++ 静态方法里也不能用 `this`,这点一致。
- `static` 入口 `main`:`public static void main(String[] args)` 是 Java 程序入口,刷题 ACM 模式里必写。

六、类与对象(刷题 80% 要写链表节点类)

Java 的 `class` $\approx$ C++ 的 `struct`:字段默认访问修饰符是 `package-private`,字段直接写在类里——比 C++ `cl`



ass 默认 private 友好得多。

6.1 刷题最小示例(必背)

链表节点(LeetCode 200+ 题的标配,这段代码背下来):

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; } // 构造器,刷题只用一个参数就够
}
```

二叉树节点(LeetCode 100+ 题的标配):

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int v) { val = v; }
}
```

6.2 C++ 对照

```
// C++ 链表节点
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v) : val(v), next(nullptr) {} // 构造时要初始化 next
};

// C++ 树节点
struct TreeNode {
    int val;
    TreeNode* left, * right;
    TreeNode(int v) : val(v), left(nullptr), right(nullptr) {}
};
```

6.3 关键差异

维度	C++ `struct`	Java `class ListNode`
字段类型	`ListNode* next` (指针)	`ListNode next` (引用)
默认访问	`public`	`package-private` (同一包可见,刷题无所谓)
构造器	`ListNode(int v) : val(v), next(nullptr) {}`	`ListNode(int v) { val = v; }` (不用初始化 next, 默认是 null)
访问成员	`node->next`	`node.next`
析构	不用(用 `delete` 手动)	不用(GC 自动)

6.4 常见疑问

• 为什么 Java 用 `class` 不用 `struct`? Java 没有 struct 关键字,只能用 class。字段不写 private 默认就是 package-private,效果跟 C++ struct 差不多。



- `new` 出来的对象在堆上,引用在栈上:跟 C++ new 一样。但 Java 永远不用 `delete`,GC 自动回收。
- 三个构造器套路(无参 / 单参 / 双参)是 LeetCode 平台生成的,刷题自己写只写一个 `(int v)` 就够。
- 为什么不需要 `next = null`? Java 引用类型字段默认初始化为 `null`,C++ 指针不会自动初始化(野指针)。Java 这点比 C++ 安全。

6.5 LeetCode 提交时的位置(刷题常踩)

```
// LeetCode 提交:辅助类放在 Solution 同文件,顶层或 static 修饰
class ListNode {           // 顶层类,不放 Solution 里面
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}

class Solution {
    public ListNode reverseList(ListNode head) {
        // ...
    }
}
```

详细 I/O 模式差异(ACM Main vs LeetCode Solution)见 01 第九节。

七、接口(`Comparable` / `Comparator` 是什么)

刷题时这两个接口天天见(PriorityQueue / Arrays.sort 都要传),但写法放 04 Comparator 小节。本篇只讲"是什么、什么时候用"。

7.1 核心:接口 = "约定某种能力"

类 implements 一个接口,等于承诺"我有这个能力"——Java 编译器会强制你实现接口里声明的方法。

```
// 接口:定义"能比较"的能力
interface Comparable<T> {
    int compareTo(T other);    // 负数:this < other, 0:相等, 正数:this > other
}
```

7.2 两个接口的区别

接口	含义	谁定义比较规则	典型用法
`Comparable<T>`	类自己说"我能比较"	类自己实现 `compareTo`	`Arrays.sort(arr)` 默认按元素 `compareTo` 排
`Comparator<T>`	外部说"我来比较 X 和 Y"	单独写一个比较器	`Arrays.sort(arr, comparator)`

7.3 一句话选型

- 元素本身有自然顺序(数字、字符串、Integer / String):不用管,直接 Arrays.sort。
- 自定义类要排序:两种写法都行,lambda 写法最常用(具体写法见 04 Comparator 小节)。
- 要多重排序方式(学生按分数 / 按姓名):用 Comparator,因为 Comparable 只能定义一种。

7.4 C++ 对照



```
// C++ 函数对象(functor)的概念
struct Comp {
    bool operator()(int a, int b) const { return a > b; } // 大顶堆
};
std::sort(v.begin(), v.end(), Comp()); // 传函数对象
// C++11 后,lambda 更常用:std::sort(v.begin(), v.end(), [](int a, int b){ return a > b; });
```

```
// Java:lambda 一行(具体写法 04 详讲)
Arrays.sort(arr, (a, b) -> b - a); // 降序,跟 C++ lambda 几乎一样
```

八、`char` vs `String` (C++ 选手易混)

C++ 里 char 和字符串混用无所谓,Java 严格区分:'a' 是 char(基本类型),"a" 是 String(对象),不能混用。

8.1 核心差异

写法	类型	备注
<code>'a'</code>	<code>`char`</code> (基本类型,2 字节,Unicode 码点)	单引号,单字符
<code>"a"</code>	<code>`String`</code> (对象)	双引号,任意长度
<code>'ab'</code>	编译报错	字符字面量只能一个字符
<code>""</code>	<code>`String`</code> (空串)	合法,长度 0

8.2 数字 ↔ 字符

```
char c = '9';
int n = c - '0'; // 9,利用 ASCII 偏移:'9'(57) - '0'(48) = 9
int n2 = c - 'a'; // 字符位置(0-25),常用于字典序
```

```
// C++ 写法一样,但 c 是 char,n 是 int,混用不会报错
char c = '9';
int n = c - '0';
```

8.3 取字符串的第 i 个字符(必记)

```
String s = "hello";
char c = s.charAt(i); // 返回 char,不是 String
// s[i] // 错误!String 没有 [] 运算符
```

8.4 遍历字符串(C++ 选手必踩)

```
// 错误!Java String 不能直接 for-each(char)
String s = "hello";
for (char c : s) { // 编译报错!String 不是 Iterable<char>
    System.out.println(c);
}
```



```
// 正确:先转成 char[]
String s = "hello";
for (char c : s.toCharArray()) { // "hello" → ['h','e','l','l','o']
    System.out.println(c);
}

// 正确 2:用 charAt 配合下标
for (int i = 0; i < s.length(); i++) {
    char c = s.charAt(i);
    System.out.println(c);
}
```

```
// C++ 范围 for 可以直接遍历 string
std::string s = "hello";
for (char c : s) std::cout << c; // 合法
```

8.5 字符串拼接

```
// char + char + char:会先自动转成 int,做加法
char a = '1', b = '2';
System.out.println(a + b); // 99(不是 "12",而是 '1'(49) + '2'(50))
System.out.println("" + a + b); // "12"(加空串触发 String 拼接)
```

8.6 记忆口诀

单引号 char,双引号 String,取字符用 `charAt`,遍历先 `toCharArray`。

九、异常处理最小化(ACM 模板必加 `throws`)

刷题只学一个异常关键词: `throws IOException`。其他 try-catch-finally 一律跳过,工程向内容不在速成范围。

9.1 为什么必须写

Java 把"可能出错的 I/O 操作"标成受检异常(checked exception),BufferedReader.readLine()、br.close() 都会抛 IOException。不写 `throws IOException` 编译直接报错。

C++ 选手第一次写 Java 会在这一步卡住:C++ 没这个语法,不写也能跑Java 不写就编译失败。

9.2 ACM 模式完整骨架(背下来)

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException { // ★ 必加
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        // ... 业务逻辑
    }
}
```



详细 I/O 模板(StreamTokenizer / PrintWriter / StringTokenizer)见 01 输入输出篇第六节"异常声明"和第七节"ACM 模式完整骨架 v2"。

9.3 LeetCode 模式不写

LeetCode 平台用 try-catch 把 IOException 包好了,提交时不用写 throws:

```
// LeetCode 模式
class Solution {
    public int[] twoSum(int[] nums, int target) {
        // 不用 throws IOException
        return new int[0];
    }
}
```

9.4 C++ 对照

```
// C++ 没有内建异常(标准库抛出,但不强制捕获)
// std::cin / std::ifstream 失败靠返回流状态判断
ifstream f("a.txt");
if (!f.is_open()) { /* 错误处理 */ }
```

其他异常(NullPointerException / ArrayIndexOutOfBoundsException / ClassCastException 等)刷题场景基本不碰——它们是运行时异常,Java 编译器不强制你处理。遇到了就是代码写错,改代码就行。

十、for-each 增强 for 循环

Java 的 for-each 语法跟 C++ 范围 for 几乎一样,但有个关键差异:Java 的 `String` 不能直接 for-each。

10.1 基本语法

```
for (ElementType x : collection) {
    // 用 x
}
```

10.2 遍历数组

```
int[] arr = {1, 2, 3, 4, 5};
for (int x : arr) {
    System.out.println(x);
}
```

```
// C++ 完全一样
std::vector<int> v = {1, 2, 3, 4, 5};
for (int x : v) std::cout << x;
```

10.3 遍历 ArrayList(见 03)



```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3));
for (int x : list) {
    System.out.println(x);
}
```

10.4 遍历字符串(注意:要先`toCharArray`)

```
String s = "hello";
for (char c : s.toCharArray()) { // 必须先转 char[]
    System.out.println(c);
}
```

10.5 坑点:遍历时不能`remove`(抛`ConcurrentModificationException`)

```
// 错误:边遍历边删
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
for (int x : list) {
    if (x % 2 == 0) list.remove(Integer.valueOf(x)); // 抛 ConcurrentModificationException
}

// 正确 1:用迭代器
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    int x = it.next();
    if (x % 2 == 0) it.remove();
}

// 正确 2:倒着遍历(下标可控)
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i) % 2 == 0) list.remove(i);
}
```

这个坑在 06 第 4 坑会再出现。

10.6 C++ 对照

```
// C++ 范围 for + remove 也有同样的坑,迭代器失效
for (auto it = v.begin(); it != v.end(); ) {
    if (*it % 2 == 0) it = v.erase(it);
    else ++it;
}
```

十一、章节小结(看完你得会)

必会项	重要度
引用用`&`访问成员,没有`*``&``->`	★★★
字符串不可变,改字符串用`StringBuilder`(API 见 04)	★★★
数组定长,长度`arr.length`(无括号),动态用`ArrayList`	★★★



<code>`final int[] a`</code> 引用不可变,内容可变	★★
<code>`static`</code> 表"类共享", <code>`static ListNode head`</code> 全局链表头常见	★★
<code>`class ListNode`</code> / <code>`class TreeNode`</code> 必背(刷题 80% 要写)	★★★
<code>`Comparable`</code> (自身有顺序)vs <code>`Comparator`</code> (外部定顺序)	★★
<code>`a`</code> 是 <code>`char`</code> , <code>`"a"`</code> 是 <code>`String`</code> ,取字符用 <code>`charAt`</code>	★★★
ACM 模式 <code>`main`</code> 必须 <code>`throws IOException`</code>	★★★
字符串遍历先 <code>`toCharArray`</code> ,遍历时不能 <code>`remove`</code>	★★

装箱小框(为什么 `ArrayList<Integer>` 必须用 `Integer` 而不是 `int`) + `Integer` 缓存池陷阱在 03 第一节,先翻容器小框就能看到。

lambda 写法全集(本篇没讲)在 04 `Comparator` 小节。

练习建议

- 手写 `class ListNode` 和 `class TreeNode` 各 5 遍,直到能闭眼写
- 写一道"字符串反转"(LeetCode 344),用 `StringBuilder.reverse()` 和 `char[]` 两种方法对比
- 写一道"最长公共前缀"(LeetCode 14),体会 `charAt` / `toCharArray` 用法

下一步

打开 03_容器与STL对照.md,学 13 个容器对照(深讲 6 / 简讲 7),这是核心章节。

